



ATRIUM

Open Metaverse Platform

Project Update

An open multiplayer 3D world platform built on glTF and WebSockets

A starter kit for a metaverse for everyone, that nobody controls — fully built on open standards, all open source. Now spanning eight packages, 311 tests, and a full pointer/interaction stack.

glTF 2.0

WebSockets

Three.js

Node.js

glTF-Transform

ES Modules

Session 35 · May 2026

github.com/tparisi/atrium

The Browser Model

Core philosophy — unchanged since day one

Navigate & Load

A world is a URL. No account required to view.
No app to install.

Static First

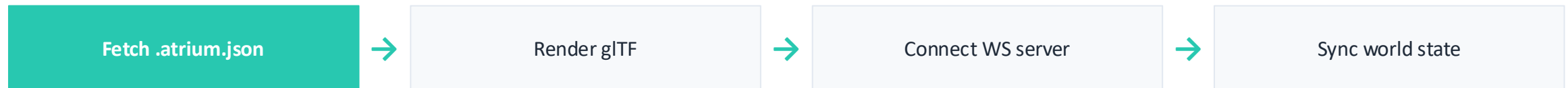
glTF renders immediately. Server connection
is optional. Graceful degradation built in.

Multiplayer Second

WebSocket layers on top. World is visible
before server connects. Offline = scene still
works.

"Atrium should feel like a browser, not a platform. You navigate to a world, it loads, you're there."

Load Sequence



Architecture

Three layers — content, protocol, runtime

Content Layer

Standard glTF 2.0

- ▶ A world is a .gltf file
- ▶ Any glTF viewer can render it
- ▶ Atrium metadata in extras.atrrium
- ▶ name · description · navigation · background

Protocol Layer

SOP — Scene Object Protocol

- ▶ JSON over WebSocket, Ajv validated
- ▶ Defined in @atrium/protocol
- ▶ som-dump on connect carries full state
- ▶ send/set echoed; view fire-and-forget

Runtime Layer

SOM + Client + Renderer-Three

- ▶ SOM — DOM-inspired API over glTF-Transform
- ▶ Symmetric: same SOM on client & server
- ▶ AtriumClient is renderer-neutral
- ▶ @atrium/renderer-three is the Three.js seam

glTF is world state. Serialize the SOMDocument and you have serialized the world — geometry, metadata, animation playback state, and ephemeral avatars all round-trip through som-dump.

SOP — Scene Object Protocol

JSON over WebSocket · Ajv-validated · every message carries seq

CLIENT → SERVER

hello	Initiate session
ping	Latency check
send	Mutate node field — broadcasts set
add	Add a new node to the world
remove	Remove a named node
view	Broadcast observer position & state

SERVER → CLIENT

hello	Session established
pong	Response to ping
tick	Heartbeat at negotiated interval
set	Authoritative world state update
add / remove	Node lifecycle broadcast
join / leave	Peer presence
view	Peer observer state relay
som-dump	Full SOM state on connect
error	Code + message

Key semantics: send/set echoed to sender · view fire-and-forget, last-write-wins · set targets any named SOM object via getObjectByName

SOM — Scene Object Model

DOM-inspired API over glTF-Transform · symmetric server + client · 109 tests

Object Hierarchy

SOMDocument	Root — extras, factories, lookups
SOMScene	addChild / removeChild
SOMNode	translation, rotation, scale, visible
SOMMesh	primitives, weights
SOMPrimitive	mode, material
SOMMaterial	PBR — baseColor, metallic, alpha
SOMCamera	perspective / orthographic
SOMAnimation	play / pause / stop, playback object
SOMTexture / SOMSkin	read-only in v0.1

Key Features

Mutation events

Every setter fires a 'mutation' event. Zero-cost when no listener. Drives client→server sync and renderer reconciliation.

Global namespace

Single `_objectsByName` Map. `som.getObjectByName('Crate') → SOMNode`.
`som.getObjectByName('WalkCycle') → SOMAnimation`. Uniform protocol routing.

Pointer dispatch

SOMObject also routes pointer events. `pointerover/out/down/up/move/click`. Leaf-only today; bubbling on the design backlog.

Wrapper caching

Stable identity: `getNodeByName('x') === getNodeByName('x')`. Full graph built once; O(1) lookups thereafter.

`som.getObjectByName('Crate').translation = [1, 0, 2]` → mutation event → set message → broadcast

One uniform write path: SOM setter → mutation event → AtriumClient → wire → server → broadcast → peer SOMs reconcile.

Animation, Avatars & Navigation

Three of the four headless controllers in @atrium/client

SOMAnimation

Wraps glTF-Transform Animation with multiplayer-aware playback state.

Playback compound property

```
{
  playing: false,
  paused: false,
  loop: false,
  autoStart: false,
  timeScale: 1.0,
  startTime: 0,
  startWallClock: null,
  pauseTime: null
}
```

One write to playback fires one mutation → one set message → atomic state transition.

AvatarController

Local + peer avatar lifecycle.

- ▶ Avatar nodes are regular SOM nodes — ephemeral per session, stamped extras.atrium.ephemeral = true
- ▶ Session identity = avatar node identity. displayName = User-\${sessionId.slice(0,4)}; node name equals displayName
- ▶ Apps own geometry — passed as a descriptor to client.connect(); AtriumClient stamps the name
- ▶ Reacts to peer:join / peer:leave / peer:view to mount, dismiss, and update peers

NavigationController

First-person navigation; broadcasts observer state via view.

Modes

WALK

ORBIT

FLY

TELEPORT

shipped / planned

- ▶ view message: position, look, move, velocity (+ optional up) — fire-and-forget, last-write-wins
- ▶ NavigationInfo authored in extras.atrium.navigation: mode array, terrainFollowing, speed range, updateRate
- ▶ WALK + ORBIT shipped; FLY, terrain-following, gravity, dead-reckoning are deferred

Pointer & Interaction

Sessions 32–35 · layered headless dispatch with policy at the top

Dispatch pipeline

DOM PointerEvent

browser native



PointerInputBridge

@atrium/renderer-three



client.dispatchPointerEvent

@atrium/client — capture, hover



SOMObject._dispatchEvent

@atrium/som — leaf dispatch



Listener handlers

app / @atrium/interaction

Packages

@atrium/renderer-three

Session 34

Three.js seam. PointerInputBridge owns hit-testing, NDC math, walk-up-by-name, and capture-coexistence. Constructor + dispose — entire surface. 19 tests.

@atrium/interaction

Session 35

User interaction policy. Selection model — nearestNonMeshAncestor, leafOnly, resolveSelectionRoot. The lamp problem solved without bubbling. 9 tests.

AtriumClient pointer API

Session 32

dispatchPointerEvent · setPointerCapture · releasePointerCapture · hasPointerCapture.
Renderer-neutral; no Three.js leaks through event detail.

Walk before run: leaf-only dispatch · mouse only · no rotate/scale · no bubbling — each a deliberate "add when we have a use case."

Repository Layout

pnpm workspaces · ES modules · no TypeScript, no build step

packages/

@atrium/protocol	SOP message schemas + Ajv validator
@atrium/som	Scene Object Model — DOM-inspired API
@atrium/server	WebSocket world server
@atrium/client	AtriumClient + controllers, headless
@atrium/interaction	User interaction policy — selection model
@atrium/renderer-three	Three.js glue — bridge, drag-math, hit-test
@atrium/gltf-extension	ATRIUM_world extension [coming]

apps/

client	Browser UI — Three.js viewport, navigation, avatars
playground	Pointer-event test bench (Session 34)

tools/

protocol-inspector	Single-file interactive protocol debugger
som-inspector	SOM tree, property sheet, world info, animations

311

tests across all packages

protocol 46 · SOM 109 · server ~38
client 96 · renderer-three 19 · interaction 9

What's Been Built

Status as of Session 35 · May 2026

Protocol & SOM

- ▶ 14 SOP message types, full Ajv validation
- ▶ SOM full object hierarchy + mutation events
- ▶ Global namespace via getObjectByName
- ▶ som-dump on connect — full state transfer

Server

- ▶ Session lifecycle, presence, broadcast
- ▶ External reference resolution at startup
- ▶ .atrium.json consumption
- ▶ som-dump filtering for external nodes

Client (headless)

- ▶ AtriumClient + Avatar/Nav/Animation controllers
- ▶ Loopback prevention + wallclock-derived seek
- ▶ peerCount with local-avatar exclusion
- ▶ Pointer dispatch + capture + hover state

Animation Arc (27–31)

- ▶ Full playback state machine on SOMAnimation
- ▶ autoStart authoring field, peer-aware
- ▶ Renderer reconciliation via replayPlaying...
- ▶ Natural completion sync via mixer 'finished'

Pointer Arc (32–34)

- ▶ AtriumClient pointer API — capture, hover, click
- ▶ PointerInputBridge in @atrium/renderer-three
- ▶ Three call sites: client, inspector, playground
- ▶ World-plane drag in the SOM Inspector

Interaction (35) ★

- ▶ @atrium/interaction package launched
- ▶ Selection model — nearestNonMeshAncestor
- ▶ leafOnly + resolveSelectionRoot policies
- ▶ Pattern C: heuristic + Alt-descend in apps

Next: Drag UX polish · pointer-events polish · pointer-event bubbling design · ATRIUM_interactivity extension · ATRIUM_world extension formalization

Design Principles

13 invariants · never violated

1

Design before code

Every session starts from a settled brief.

2

No throwaway code

Every line tested against real implementation.

3

Incremental correctness

Each layer fully working before the next is built on top.

4

glTF on the wire

Protocol carries glTF node descriptors directly.

5

Server is policy-free on geometry

Server never constructs or interprets meshes.

6

AtriumClient is geometry-agnostic

Avatar descriptor is opaque to the client.

7

SOM is the source of truth

All world state mutations go through the SOM.

8

Static first, multiplayer second

Client renders even if the server is unreachable.

9

glTF is world state

Serialize the Document → world is serialized.

10

packages/client is headless

No browser globals — portable to Node, bots.

11

Renderers reconcile to current state

Walk the SOM when ready; don't trust event order.

12

packages/client is renderer-neutral

Bridge pattern is the single seam where Three.js meets the client.

13

Interaction policy lives in @atrium/interaction

Mechanism in core; policy composes on top. Apps consume directly.

Principles 12 and 13 emerged from the pointer/interaction arc — abstraction shape validated by three real consumers.



A T R I U M

github.com/tparisi/atrium

Where We Are

Session 35 complete · Session 36 open

35

sessions

8

packages

311

tests

6

test fixtures

Recent landings

- ▶ Session 35 — @atrium/interaction package + selection model
- ▶ Session 34 — @atrium/renderer-three + PointerInputBridge
- ▶ Session 33 — Inspector click-to-select + drag-to-translate
- ▶ Session 32 — pointer event dispatch + capture + hover
- ▶ Sessions 27–31 — full animation arc with connect-time reconciliation

Browser, not platform. Open standards. All open source.